

App Development Project Report

App Dev Project Report

1. Student Details

Name: Meenakshi K

Roll Number: 24F2001170

Email: 24f2001170@ds.study.iitm.ac.in

About Me: I am a student of the IIT Madras BS Degree Programme in Data Science with a strong interest in astrophysics and programming. I am particularly interested in using computational methods and data analysis to study problems in astronomy and space science. I enjoy learning new concepts through hands-on projects and applying programming to scientific problems.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

The aim of this project is to develop a web-based Trekking Management Application that allows trekking organisations to manage treks, staff members, users and bookings through a single application.

The application provides different functionality based on the role of the user:

- **Admin:** manages treks, users, staff, bookings and application settings.
- **Staff:** manages assigned treks and their participants.
- **User/Trekker:** browses available treks, makes bookings and manages their bookings.

The application also maintains booking information and trek availability through a relational database.

Approach:

The application was developed using Flask as the backend framework and SQLite as the database. SQLAlchemy is used to interact with the database through Python models.

The application uses role-based access control to provide different dashboards and functionality to Admin, Staff and User accounts.

The frontend was developed using HTML, Jinja2 templates and Bootstrap 5. Forms are used for actions such as registration, login, trek creation, editing trek information, booking and cancellation.

The application also implements password hashing and session-based authentication.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist with parts of the Flask backend logic, routing, and request handling.

The extent of AI/LLM usage is around 15-20%, limited to code suggestions and documentation formatting.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Python	Main programming language
Flask	Backend web framework and routing
Flask-SQLAlchemy / SQLAlchemy	ORM and database interaction
SQLite	Database for storing users, treks and bookings
HTML5	Structure of web pages
Jinja2	Dynamic HTML templating
Bootstrap 5	Responsive frontend styling
Werkzeug	Password hashing and authentication-related utilities
Flask Sessions	Maintaining logged-in user sessions

5. Database Schema / ER Diagram

Tables:

1. **User**: stores user account details (id, name, email, password_hash, role, contact, is_approved, is_blacklisted)
2. **Trek**: stores trek/expedition details (id, name, location, difficulty, duration, total_slots, available_slots, assigned_staff_id, status, start_date, end_date, description)
3. **Booking**: logs trek bookings made by users (id, user_id, trek_id, booking_date, status)

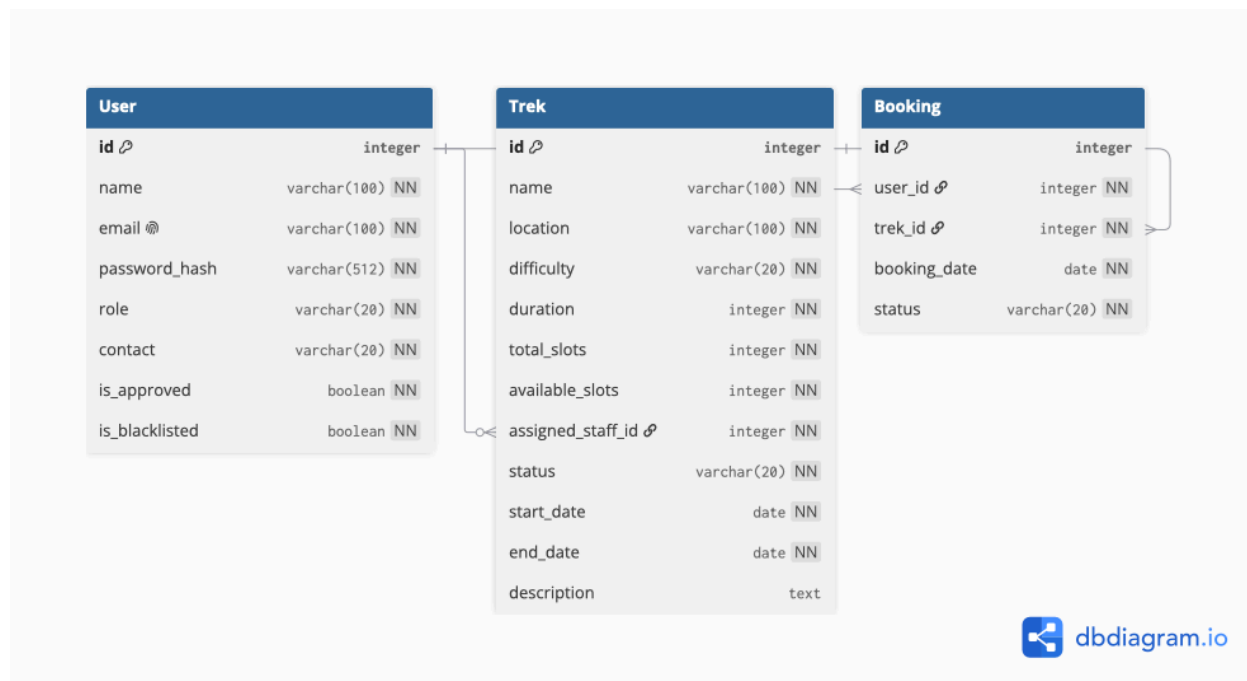
Relationships:

- One-to-Many → **User** → **Trek** (a User, as staff, can be assigned to many Treks via assigned_staff_id)
- One-to-Many → **User** → **Booking** (a User can make many Bookings)
- One-to-Many → **Trek** → **Booking** (a Trek can have many Bookings)

Interpretation:

- A User can make multiple Bookings.
- A Trek can have multiple Bookings.
- A Staff User can be assigned to multiple Treks.
- Each Booking belongs to one User and one Trek.

ER diagram:



(image created using dbdiagram.io)

6. Application Architecture

The project follows a Flask-based web application structure.

```
.
├── app.py
├── instance
│   └── trekking.db
├── models.py
├── README.md
├── report.pdf
├── requirements.txt
├── templates
│   ├── add_treks.html
│   ├── admin_bookings.html
│   ├── admin_dashboard.html
│   ├── admin_report.html
│   ├── admin_settings.html
│   ├── booking_history.html
│   ├── browse_treks.html
│   ├── change_password.html
│   ├── edit_profile.html
│   ├── edit_trek.html
│   ├── index.html
│   ├── login.html
│   ├── manage_staff.html
│   ├── manage_treks.html
│   ├── manage_users.html
│   ├── my_bookings.html
│   ├── participants.html
│   ├── register.html
│   ├── staff_dashboard.html
│   ├── staff_my_treks.html
│   ├── staff_participants.html
│   ├── staff_treks.html
│   ├── staff_update_trek.html
│   ├── update_trek.html
│   ├── user_dashboard.html
│   ├── view_staff_trek.html
│   └── view_trek.html
```

The main application logic and routes are handled through `app.py`, while database models are defined in `models.py`. HTML templates are stored in the `templates` directory.

7. User Roles and Features

The application provides different functionality for each user role:

- **Admin:** Manages treks, staff, users and bookings; approves staff accounts, manages blacklists, views reports and manages account settings.
- **Staff:** Views assigned treks, updates trek status and availability, and manages trek participants.
- **User/Trekker:** Browses and filters treks, views trek details, books and cancels treks, views booking history and manages their profile.

Role-based access control ensures that each user can only access the features relevant to their role.

8. Authentication and Access Control

The application uses session-based authentication.

When a user logs in successfully, their user ID is stored in the session. Each protected route checks the logged-in user's role before allowing access.

For example:

- Admin routes require `role === 'admin'`
- Staff routes require `role === 'staff'`
- User routes require `role === 'user'`

Staff accounts also require approval from the Admin before they can access staff functionality.

Passwords are stored using password hashing instead of plain text.

Also, the application checks ownership for staff operations. For example, a staff member cannot access or modify a trek assigned to another staff member.

9. Testing

The application was tested using separate Admin, Staff and User accounts.

The following workflows were tested:

- User registration and login
- Admin trek, user and staff management

- Staff approval and access control
- Staff trek and participant management
- User trek search and filtering
- Trek booking and cancellation
- Booking history
- Profile editing
- Password change
- Role-based access restrictions

After testing, the application was also tested using a fresh database containing only the Admin account to verify the complete workflow from registration and staff approval to trek creation and user booking.

9. Video Presentation

Drive Link:

<https://drive.google.com/file/d/15VICM2RXkWu1esfySzP6UQjluWCu5bx5/view?usp=sharing>

(Accessible to all with “View” permission.)
